

알고리즘 프로젝트 최종 보고서



[알고리즘 프로젝트]			
과목명	알고리즘(06)	담당교수	주종화 교수님
제출일	2024/12/02	팀명	우고리즘(13 조)
팀원		학번	역할
우지민		2021111642	팀장, BWT-SW 알고리즘
박승우		2021112468	보고서 제작, DeBruijn 그래프 알고리즘
안성우		2020112029	발표자료 제작, Greedy overlap 알고리즘

목 차

1. 프로젝트 목표	3
1.1. 프로젝트 목표	3
1.2. 프로젝트 구성	3
2. 데이터 선정	3~4
2.1. 초기 데이터 선정	3
2.2. 데이터 크기 조정	3~4
3. 알고리즘 설명	4~19
3.1. DeBruijn Graph Algorithm	4~8
3.2. BWT-SW Algorithm	9~14
3.3. Greedy Overlap Algorithm	14~19
4. 결과 비교 및 분석	20~22
4.1. 알고리즘별 시공간 복잡도	20~21
4.2. 실제 프로그램 작동 시 비교	22
5. 결론	23
6. 참고문헌	24

1. 프로젝트 목표

1.1. 프로젝트 목표

짧은 DNA 서열 조각들(Short Reads)을 조합하여 원본 서열(Original Sequence)을 재구성하는 것이 본 프로젝트의 목표이다. 고려할 수 있는 요소들로는 다음과 같다.

- 원본 서열의 길이 N
- Short Read의 개수(블록의 개수) M
- Short Read의 길이 L
- 서열 중복 및 불일치 존재 가능성

본 팀은 다양한 알고리즘을 비교 분석하여 최적의 서열 재구성 방식을 찾고자 한다. 이를 위해 DeBruijn Graph, BWT-SW, Greedy Overlap 총 세 가지 알고리즘을 설계하고 비교 및 분석한다.

1.2. 프로젝트 구성

프로젝트는 다음과 같은 내용으로 구성되어 있다.

- 알고리즘 클래스 3개 (DeBruijn Graph, BWT-SW, Greedy Overlap Algorithm)
- 알고리즘의 실행 전역함수 3개 (do_debruijn, do_BWT_SW, do_greedy_overlap)
- 보조 클래스 3개 (SequenceMutator, ShortBlockGenerator, ResultAnalyzer)
- 보조 전역 함수 2개 (handleException, printResult)
- 전역 변수 5개 (K, L, M, N, J)

세부적으로는 Main 함수로 3가지, do_debruijn, do_BWT_SW, do_greedy_overlap 함수가 존재하고, 각 함수는 클래스로 구현된 DeBruijn, BWT-SW, Greedy Overlap 알고리즘을 실행한다. 보조 클래스는 각각이 원본 서열로부터 돌연변이를 생성하여 우리의 재구성 시퀀스에 오차를 추가하는 SequenceMutator 클래스, 블록을 생성하는 ShortBlockGenerator, 유사도 및 결과를 분석하는 ResultAnalyzer 클래스이다. 이들의 생성에는 전역변수 K (돌연변이의 개수), L (블록의 길이), M (블록의 개수), N (원본/복원 시퀀스의 길이), J (블록간 중복되는 길이)로 그 꼴을 조정한다.

2. 데이터 선정

2.1. 초기 데이터 선정

임의의 랜덤 서열보다는 신뢰할 수 있는 실제 유전자 데이터를 사용하는 것이 더 적합하다고 판단하여 미국 국립생물공학정보센터(NCBI: National Center for Biotechnology Information) 웹사이트의 데이터를 참고하였다. (<https://www.ncbi.nlm.nih.gov/>).

선택한 데이터는 DMD(듀센 근이영양증) dystrophin [Homo sapiens (human)] 유전자로, 약 200만 염기쌍 길이를 가지는 AGCT 서열이다. 이는 디스트로핀이라는 단백질의 사람 유전자 데이터로, 다양한 유전자 분석 연구에 활용되는 신뢰할 수 있는 정보이다. (데이터 출처: [NCBI Gene ID: 1756](#))

2.2. 데이터 크기 조정

초기에 설계된 200만 염기쌍 데이터를 그대로 사용했을 때, 알고리즘 구현 과정에서 결과를 확인하는 데 시간이 너무 오래 걸리는 문제가 발생하였다. 이에 따라 데이터 크기를 다음과 같이 조정하여 실험을 진행하였다.

2.2.1. 테스트 구현용 데이터

알고리즘을 구현하고 테스트하는 과정에서 200 만 염기쌍 데이터의 일부를 추출하여, 약 5,000 개의 염기쌍으로 구성된 테스트 구현용 데이터를 생성하였다. 이 데이터를 통해 알고리즘의 구현 과정에서 결과값을 빠르게 확인할 수 있었다.

2.2.2. 최종 성능 평가용 데이터

알고리즘 구현을 성공적으로 완료한 후, 충분히 큰 데이터로 확장하여 최종 결과를 도출하였다. 최종 실험은 약 6 만 염기쌍 크기의 데이터에서 수행되었으며, 이 데이터 또한 처음 데이터로 선정한 DMD dystrophin 유전자의 일부를 추출한 데이터이다.

이와 같은 단계적 접근 방식을 통해 구현과 분석의 효율성을 높이고, 신뢰성 있는 결과를 얻을 수 있었다.

3. 알고리즘 설명

3.1. DeBruijn Graph Algorithm

3.1.1. 작동 원리

DeBruijn 그래프는 DNA 시퀀스를 조각화(k-mer)하고, 이를 기반으로 그래프를 생성하여 서열을 복원하는 방법이다. k-mer 이란, 길이가 k 인 연속적인 서열 조각들을 말하고 각 Short Read 를 슬라이딩 윈도우하여 생성한다.

Ex) 원본 서열이 ACGTTGCA 이고 k=4 라면, k-mer 은 ACGT, CGTT, GTTG, TTGC, TGCA 임.

3.1.2. 구성 요소

DeBruijn 은 그래프 형태로, 노드(Nodes)와 간선(Edge)을 구성 요소로 가지고 있다. k-mer 의 prefix(접두사)와 suffix(접미사)가 각각 노드가 되고, 간선은 k-mer 전체를 표현하며 한 노드에서 다른 노드로 연결된다.

Ex) k-mer ACGT 에서 노드는 prefix 인 ACG 와 suffix 인 CGT 이고, 간선은 ACG -> CGT 로 나타냄.

3.1.3. 알고리즘의 흐름

① k-mer 생성

입력된 서열 조각들(Short Reads)에서 k 길이의 k-mer 들을 생성한다. 각 k-mer 은 prefix(처음 k-1 개 문자)와 suffix(마지막 k-1 개 문자)로 나뉜다.

② DeBruijn 그래프 생성

각 k-mer 을 기반으로 노드와 간선을 생성한다. prefix는 출발 노드, suffix는 도착 노드가 되며, 이 과정에서 동일한 prefix 와 suffix 를 가진 k-mer 들은 하나의 간선으로 합쳐진다. 이때, threshold 개념을 추가해 어느 정도 오차를 허용한다.

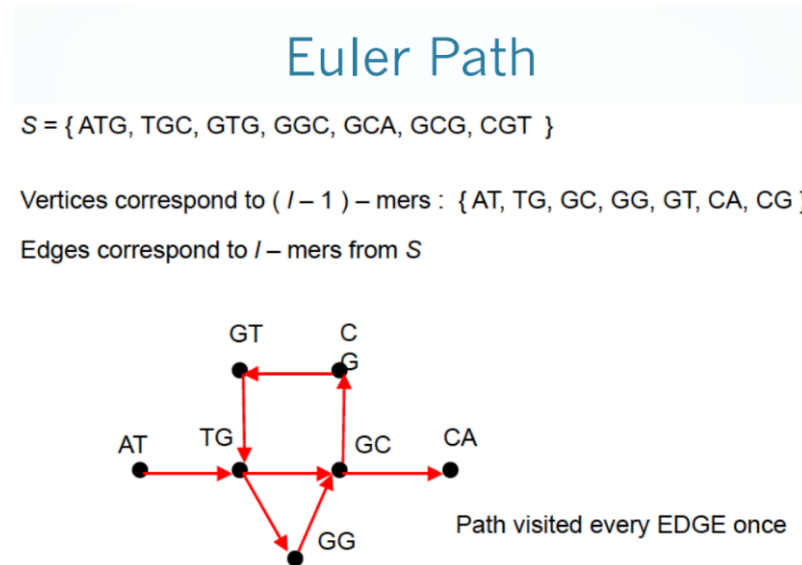
③ Eulerian Path 찾기

Eulerian Path 란 모든 간선을 방문하여 경로를 찾는 방식을 말한다. 먼저, 시작 노드와 끝 노드를 찾아 정하고, 스택을 사용해 깊이 우선 탐색(DFS) 방식으로 Euler Path 를 찾는다.

④ 서열 재구성

Euler Path 를 따라 노드를 방문하면서 겹치는 부분을 제거하여 최종 서열을 재구성한다.

전체적인 과정이 아래의 [그림 1]과 같이 수업시간에 배운 내용과 거의 유사하다.



[그림 3.1-1] Euler Path, DeBruijn Graph 알고리즘의 원리

3.1.4. 코드 적용

① k-mer 및 DeBruijn 그래프 생성

```
void buildGraph(const std::vector<std::string>& reads, int k, int threshold) { ... }
```

reads(짧은 서열 조각들)에서 길이가 k 인 k -mer 을 생성하고, 이를 기반으로 노드(prefix 와 suffix)와 간선을 만든다. 그리고, threshold 라는 개념을 추가해 threshold 로 설정한 수만큼의 오차까지는 허용한다. 즉, 염기서열에 돌연변이가 발생하여 원본 시퀀스와 살짝 다른 경우를 고려하여 유사한 서열을 동일한 간선으로 처리할 수 있도록 한 것이다.

② Eulerian Path 찾기

```
std::vector<std::string> findEulerianPath() { ... }
```

먼저, 그래프의 각 노드의 진입 차수와 진출 차수를 계산하여 시작 노드와 끝 노드를 정한다. 시작 노드는 진출 차수(Out-degree)가 진입 차수(In-degree)보다 1 이 크고, 끝 노드는 반대로 진입 차수가 진출 차수보다 1 이 큰 점을 이용하여 찾는다. 시작 노드부터 스택을 사용해 DFS(깊이 우선 탐색) 방식으로 경로를 탐색하고, 모든 간선을 소모할 때까지 간선을 하나씩 제거해가며 방문한다.

③ 서열 재구성

```
std::string reconstructSequence(const std::vector<std::string>& reads, int k, int threshold) { ... }
```

Euler Path 를 따라가며 k -mer 의 겹치는 부분을 제외한 뒷부분의 한글자씩 추가해가며 최종 서열을 만든다. 이때, 알고리즘 시작 시간과 종료 시간을 기록하여 알고리즘 수행 시간을 계산하여 출력한다.

3.1.5. 알고리즘의 장점

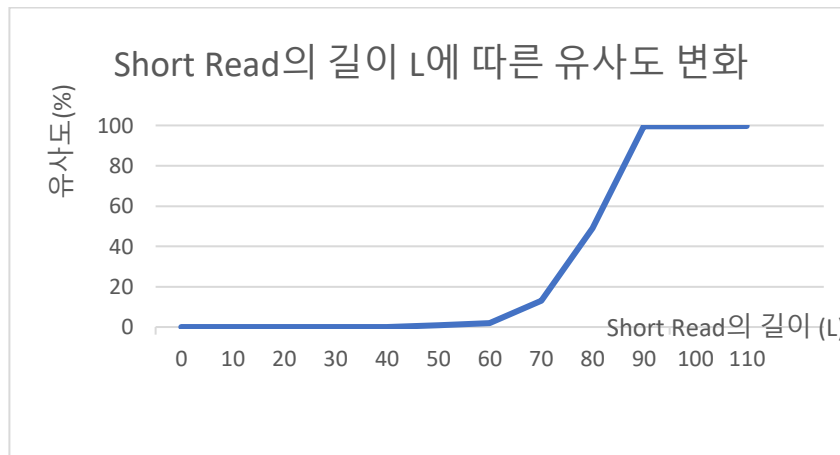
- ① 효율적인 메모리 사용: 중복된 k-mer 을 하나의 노드와 간선으로 처리해 메모리 사용량을 줄인다.
- ② 빠른 처리 시간: unodreded_map 을 사용해 map 보다 더 빠른 탐색이 가능하고 데이터가 많을 시 월등히 좋은 성능을 보인다.
- ③ 불일치(돌연변이) 처리: 돌연변이가 포함된 불완전한 서열에서도 threshold 개념을 탑재하여 어느 정도 오차를 허용한다. Ex) 길이가 50 인 서열에서 threshold 를 5 로 설정하면, 50 자의 AGCT 염기쌍 중 45 자 이상만 일치하면 동일한 간선으로 처리함.
- ④ 중복 처리: 중복된 k-mer 을 하나의 노드로 처리하고 Out-degree 와 In-degree 를 가중치(weight)처럼 사용하여 동일한 서열이 여러 번 나와도 처리가 가능하다.

3.1.6. 주요 데이터 값에 따른 결과 비교

데이터 값 설정에 따른 큰 오차율이 발생할 수 있다. Short Read 의 길이 L , 개수 M , k-mer 의 k 값 설정에 따라 유사도가 크게 차이 난다. 이는 Euler Path 를 탐색하다가 더 이상 방문할 간선이 없으면 그대로 알고리즘이 종료되기 때문이다. 그러므로, Euler Path 가 중간에 끊기지 않기 위해 데이터 값(L, M, k)을 적당히 잘 설정해 주어야 한다.

① Short Read(블록)의 길이 L

Short Read 의 길이가 클수록 생성되는 k-mer 의 수가 많아지기 때문에 유사도는 증가한다.



[그래프 3.1-1] Short Read 의 길이 L 에 따른 유사도 변화

Short Read 에서 k-mer 을 만드는 것이므로 L 의 값이 k 인 50 보다 작으면 "invalid string position" 오류가 발생한다. 그 이후, L 이 클수록 유사도가 급격히 증가하는 것을 볼 수 있으며, 특히 100 이상일 때 유사도가 99% 이상으로 활용 가능하다고 판단되었다.

<pre> int K = 10; // 돌연변이 개수 int L = 100; // 블록 길이 int N = 5000; // 파일에서 읽어올 알파벳 수 int M = 1000; // 블록의 개수 </pre>		DeBruijn Graph Algorithm 실행:
		알고리즘 수행 시간: 403ms
		매칭완료, 분석 수행
		돌연변이의 비율: 0.2%
		유사도: 99.44%

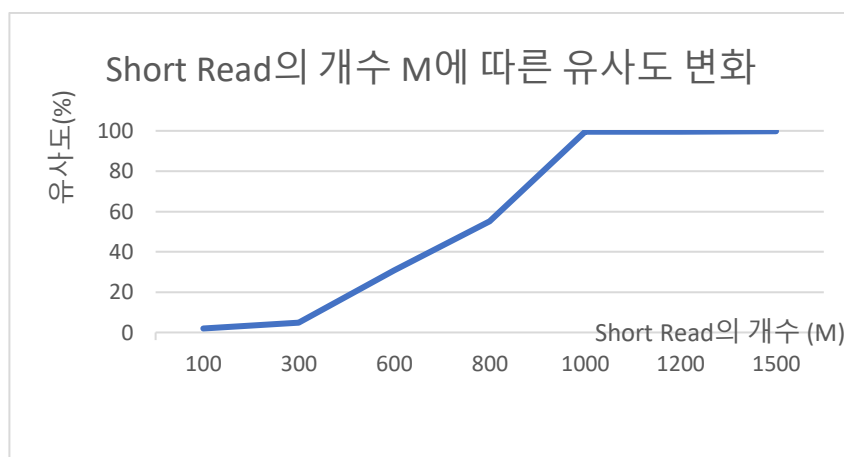
[그림 3.1-2] $L = 100$ 일 때 출력되는 유사도 값

② Short Read(블록)의 개수 M

Short Read의 개수가 많을수록 그만큼 원본 시퀀스를 복원하는데 데이터가 많아지는 것이기 때문에 유사도는 증가한다. 충분히 큰 M이 주어지지 않아 서열 중간에 빈 부분이 생기면 Euler Path를 탐색하다가 이 부분에서 경로를 찾지 못하고 종료되어 버린다. 즉, 충분히 큰 M 값이 주어지지 않는다면 DeBruijn Graph 알고리즘한테 치명적일 수 있다.

<pre>int K = 10; // 돌연변이 개수 int L = 100; // 블록 길이 int N = 5000; // 파일에서 읽어올 알파벳 수 int M = 300; // 블록의 개수</pre>		DeBruijn Graph Algorithm 실행: 알고리즘 수행 시간: 124ms 매칭완료, 분석 수행 돌연변이의 비율: 0.2% 유사도: 4.94%
<pre>int K = 10; // 돌연변이 개수 int L = 100; // 블록 길이 int N = 5000; // 파일에서 읽어올 알파벳 수 int M = 600; // 블록의 개수</pre>		DeBruijn Graph Algorithm 실행: 알고리즘 수행 시간: 217ms 매칭완료, 분석 수행 돌연변이의 비율: 0.2% 유사도: 30.84%
<pre>int K = 10; // 돌연변이 개수 int L = 100; // 블록 길이 int N = 5000; // 파일에서 읽어올 알파벳 수 int M = 800; // 블록의 개수</pre>		DeBruijn Graph Algorithm 실행: 알고리즘 수행 시간: 291ms 매칭완료, 분석 수행 돌연변이의 비율: 0.2% 유사도: 55.22%
<pre>int K = 10; // 돌연변이 개수 int L = 100; // 블록 길이 int N = 5000; // 파일에서 읽어올 알파벳 수 int M = 1000; // 블록의 개수</pre>		DeBruijn Graph Algorithm 실행: 알고리즘 수행 시간: 403ms 매칭완료, 분석 수행 돌연변이의 비율: 0.2% 유사도: 99.44%

[그림 3.1-3] M = 300, 600, 800, 1000일 때 출력되는 유사도 값



[그래프 3.1-2] Short Read의 개수 M에 따른 유사도 변화

M이 각각 300, 600, 800, 1000일 때, 유사도는 약 5%, 31%, 55%, 99%로 나온다. 이처럼 L을 100이라고 가정했을 때, M을 N(원본 서열의 길이)/5 정도로 설정해주면 Short Read들이 원본 시퀀스의 데이터를 빈 부분 없이 포함할 수 있을 것이다.

```

DeBruijn Graph Algorithm 실행:
int K = 10;    // 돌연변이 개수
int L = 100;   // 블록 길이
int N = 30000; // 파일에서 읽어올
int M = 6000;  // 블록의 개수
알고리즘 수행 시간: 2424ms
매칭완료, 분석 수행
돌연변이의 비율: 0.032175%
유사도: 99.9292%

```

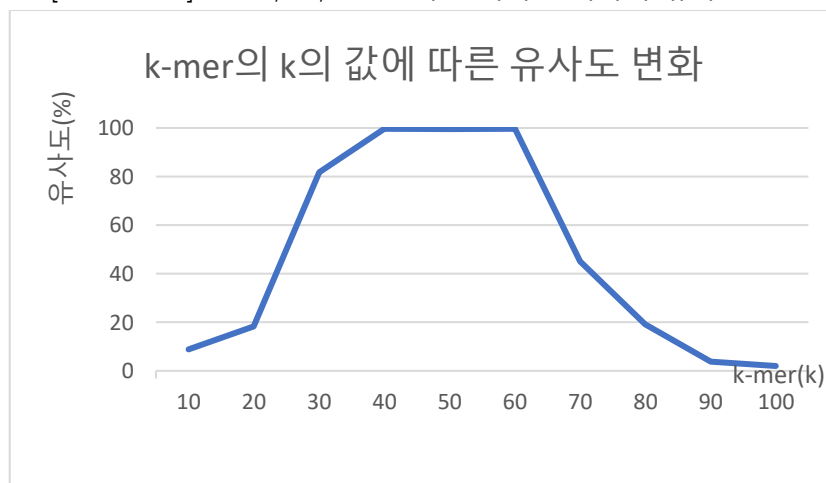
[그림 3.1-4] N = 30000 일 때 M=N/5=6000 으로 설정하면 출력되는 유사도 값

③ k-mer 에서 k 의 값

k-mer 을 생성할 때 k 를 어떤 값으로 설정하는가에 따라 결과가 크게 달라진다. 너무 작으면 Euler Path 를 찾을 때 비슷한 노드로 경로를 잘못 찾을 수 있고, 너무 길면 노드를 충분히 많이 생성하지 못해 경로를 잃어버릴 가능성이 크다. 이에 k 의 값은 $L(\text{Short Read 의 길이})/2$ 가 적당하다고 판단하여 본 프로젝트에는 k 의 값을 50 으로 설정하였다.

DeBruijn Graph Algorithm 실행:	DeBruijn Graph Algorithm 실행:	DeBruijn Graph Algorithm 실행:
알고리즘 수행 시간: 376ms	알고리즘 수행 시간: 405ms	알고리즘 수행 시간: 328ms
매칭완료, 분석 수행 돌연변이의 비율: 0.2% 유사도: 18.18%	매칭완료, 분석 수행 돌연변이의 비율: 0.2% 유사도: 99.44%	매칭완료, 분석 수행 돌연변이의 비율: 0.2% 유사도: 45.08%

[그림 3.1-5] k= 20, 50, 70 일 때 출력되는 각각의 유사도 값



[그래프 3.1-3] k-mer의 k의 값에 따른 유사도 변화

3.1.7. 알고리즘의 단점 및 개선방안

3.1.6 에서 살펴본 것처럼, De Bruijn 그래프 기반 알고리즘은 대규모 데이터를 효율적으로 처리할 수 있지만, 적절한 데이터 설정 값이 주어지지 않으면 올바르게 작동하지 않는 한계점이 있다. 이는 L, M, k 값의 부적절한 설정으로 인해 발생하며, 데이터가 충분히 연결되지 않으면 그래프의 일부 구간에서 탐색할 경로가 끊겨 알고리즘이 그대로 종료되기 때문에 발생한다. 이에 대한 개선방안으로는 다음과 같다.

경로 탐색 중 데이터가 끊기는 경우, 끊긴 지점 이후의 데이터를 탐색하여 복구를 시도하여 중단된 구간을 제외하고 탐색을 이어나가는 재탐색 알고리즘을 추가 구현하는 것이다.

또, $M=N/5$, $k = L/2$ 수준으로 데이터 값을 동적으로 설정함으로써 충분히 많은 데이터를 제공하여 서열 간 중복을 보장하는 방법도 있다.

3.2. BWT-SW Algorithm

3.2.1. 작동 원리

BWT-SW 알고리즘은 Burrows-Wheeler Transform (BWT)와 Smith-Waterman 알고리즘 (SW)을 결합하여 시퀀스 재구성을 수행하는 알고리즘이다. BWT 는 원본 문자열을 변환하여 연속적인 문자의 패턴을 압축하는 방식으로, 효율적인 검색을 위해 FM-Index 를 사용한다. FM-Index 는 주로 BWT 변환된 서열에서 walk-left 또는 backward search 를 수행할 수 있도록 구현한 방법이며, 이를 통해 빠른 문자열 탐색이 가능하다. Smith-Waterman 알고리즘은 지역 정렬을 통해 두 문자열 간의 유사도를 계산하여 탁월하게 최적의 일치치를 찾으나, 속도가 매우 느린 편이다. 이 두 기법은 서로 보완적인 역할을 하며, FM-Index 를 수행하여 빠르게 검색 후, 오차를 SW 알고리즘이 보완하는 식이다.

3.2.2. 구성 요소

BWT-SW 알고리즘은 bwt 알고리즘으로 변환되는 참조(원본)시퀀스와 bwt 처리된 시퀀스, 여기서 FM index 데이터 구조를 구성하는 C 일차원 배열, Occ 이차원 배열, 원본 시퀀스의 인덱스를 추적할 수 있는 Suffix 배열이 주 구성요소이다.

3.2.3. 알고리즘의 흐름 및 예시

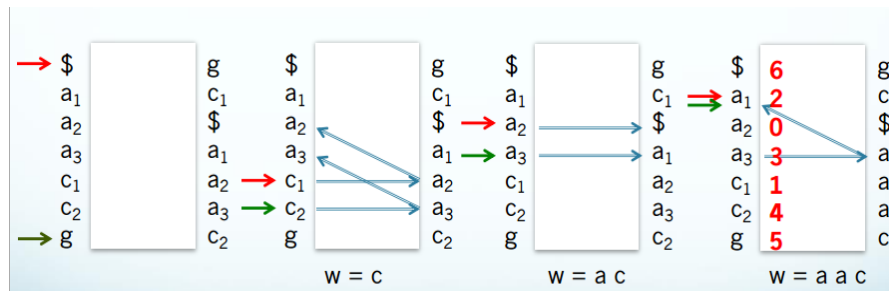
시퀀스로 원본 시퀀스 ATCGTACG\$, 블록 시퀀스 ATCG 가 있다고 하자.

- ① BWT 수행 및 FM-Index 생성: 알고리즘은 우선 원본 시퀀스를 BWT 를 통해 변환하고, 이를 기반으로 FM-Index 를 생성한다. BWT 는 원본 시퀀스에서 각 접미사를 사전식으로 정렬하여, 가장 마지막 열의 문자를 얻는다. 위의 예시로는 **T\$ATCCGAG** 이 bwt 문자열이 된다.

5	ACG\$ATCGT
0	ATCGTACG\$
6	CG\$ATCGTA
2	CGTACG\$AT
7	G\$ATCGTAC
3	GTACG\$ATC
4	TACG\$ATCG
1	TCGTACG\$A
8	\$ATCGTACG

- ② FM-Index 구축: C 배열과 Occ 배열에는 각 문자의 누적 빈도표를 생성한다. 이를 통해, LF-Mapping 을 사용하여 빠르게 특정 문자를 찾을 수 있게 된다, C 와 Occ 로는 각각 첫번째 열, 마지막 열이 저장된다고 보면 되는데, C 에는 문자의 누적 개수가, OCC 배열에는 bwt 문자열의 특정 인덱스까지의 문자의 개수가 저장된다. C 배열에는 \$:0 A:1 C:3 G:5 T:7 이 저장될 것이며, Occ 에는 \$:0 0 1 1 1 1 1 1 1, A: 0 0 0 1 1 1 1 1 2, C: 0 0 0 0 0 1 2 2 2, G: 0 0 0 0 0 0 1 1, T: 0 1 1 1 2 2 2 2 2 이 저장된다.
- ③ 문자열 검색 (Backward Search): FM-Index 를 사용해 쿼리 문자열(block)을 원본 시퀀스에서 찾는다. 쿼리 문자열은 BWT 의 마지막 열과 C, Occ 배열을 활용하여 찾는다. 구체적 너무 복잡하니 우선 생략한다. 이때, 교안의 walk-left 방식을 코드로 구현하게 되는데, 쿼리 문자열을 뒤에서부터 잘라서 찾는다. bwt 가 abc 순으로 정렬된 걸 이용하여, 쿼리의 단계별로 구간을 설정하며, 다음 단계로 넘어갈수록 구간을 줄여 나가며 특정한다. 이때 마지막 도착지 첫 열의 인덱스를 suffix array 를 이용해 원본 문자열의 인덱스로 변환한다.

[그림 3.2-1] 원본 시퀀스 ATCGTACG\$를 BWT 변환



[그림 3.2-2] 수업 중 소개된 Walk left 방식을 묘사한 그림

- ④ Smith-Waterman 알고리즘 적용: 앞의 검색에도 검색된 구간이 없으면, Smith-Waterman 알고리즘을 사용하여 원본 시퀀스에서 가장 유사한 부분을 찾아낸다. 이 알고리즘은 두 문자열의 지역 정렬을 통해 최적의 일치를 찾으며, 이를 통해 블록을 재구성된 시퀀스에 삽입한다.
- ⑤ 결과 산출: 추출한 인덱스를 기반으로 블록을 인덱스에 삽입한다.

3.2.4. 코드 적용

알고리즘은 C++로 구현되어 있으며, 핵심 기능은 BWT_SW_Algorithm 클래스에 포함되어 있다. 이 클래스는 입력된 블록 리스트와 원본 시퀀스를 받아 BWT 변환을 수행하고, FM-Index를 생성한 후, 주어진 블록들이 원본 시퀀스에서 가장 적합한 위치에 맞춰지도록 한다. 주된 함수로는 reconstructFMIndex(), applyBWT(), buildFMIndex(), searchquery(), walkleft(), applySW() 등이 있으며, 이를 통해 시퀀스를 재구성한다.

1. reconstructFMIndex 함수

```
// 1. BWT를 수행하고 FM-Index를 생성할
void reconstructFMIndex({ ... })
```

이 함수는 먼저 원본 시퀀스가 비어 있는지 확인하고, 비어 있지 않으면 applyBWT 함수를 호출하여 BWT를 계산한 후, buildFMIndex 함수로 FM-Index를 구축한다.

1-1. applyBWT 함수

```
// 1-1. Burrows-Wheeler Transform 적용
std::string applyBWT(const std::string& input)({ ... })
```

Bwt를 수행하고, suffix array를 생성한다.

1-2. buildFMIndex 함수

```
// 1-2. FM-Index에 BWT 처리된 문자열의 문자 개수를 저장
void buildFMIndex(const std::string& bwt)({ ... })
```

이 함수는 FM-Index를 구축하는 함수로, BWT 결과를 이용하여 C 배열과 Occ 배열을 생성한다. C 배열은 BWT의 첫 번째 열에서 각 문자들의 발생 빈도를 추적하며, Occ 배열은 각 문자에 대해 특정 위치까지의 발생 횟수를 저장한다.

2. searchquery 함수

```
// 2. FM-Index를 사용하여 원본 문자열에서의 위치 반환
std::vector<int> searchquery(const std::string& query){ ... }
```

이 함수는 FM-Index 를 사용하여 주어진 쿼리 문자열이 원본 시퀀스에 존재하는지 찾는 함수이다. LF-Mapping 을 사용하여 쿼리 문자열을 뒤에서부터 검색하고, 해당 위치를 원본 시퀀스에서 찾는다.

2-1. walkleft 함수

```
// 2-1. LF-Mapping 기반 backward searching
std::pair<int, int> walkleft(const std::string& query){ ... }
```

이 함수는 LF-Mapping 을 기반으로 쿼리 문자열을 왼쪽으로 이동하며 FM-Index 에서 위치를 찾는 함수이다. 쿼리 문자열을 뒤에서부터 차례대로 처리하여 매칭된 구간을 찾는다.

```
// LF-Mapping 적용
start = C[charIdx] + Occ[charIdx][start];
end = C[charIdx] + Occ[charIdx][end];
```

LF-Mapping: 쿼리 문자열의 각 문자를 처리하며, 해당 문자의 위치를 C 배열과 Occ 배열을 이용하여 추적한다. 재귀적인 계산식.

최종 구간 반환: 쿼리 문자열이 매칭되는 구간의 시작과 끝 인덱스를 반환한다.

3. applySW 함수

```
// 3. Smith-Waterman 알고리즘 적용 (지역 정렬)
int applySW(const std::string& a, const std::string& b){ ... }
```

이 함수는 Smith-Waterman 알고리즘을 사용하여 두 문자열 간의 지역 정렬을 수행한다. 주어진 두 문자열을 비교하여, 매칭 점수를 계산하고 최대 점수를 반환한다. 동적 계획법을 이용해 두 문자열을 비교하며 점수를 계산한다. 매칭일 경우에는 +2, 불일치일 경우에는 -1 의 점수를 부여하며, 가능한 모든 경로를 고려하여 최적의 점수를 찾는다. 최대 점수 반환: 최종적으로 계산된 DP 테이블에서 최대 점수를 만드는 인덱스와 그 최초 인덱스를 찾아 반환한다.

3-1. reconstructSequence 함수

```
//4. 시퀀스 재구성 함수.
const std::string& reconstructSequence(){ ... }
```

이 함수는 전체 알고리즘의 흐름을 관리하며, 주어진 블록들을 원본 시퀀스에 맞게 재구성하는 주 흐름 함수이다. 블록 리스트를 순차적으로 처리하면서 BWT 와 FM-Index 를 사용하여 각 블록의 위치를 찾고, Smith-Waterman 알고리즘을 사용하여 가장 유사한 위치를 찾아 해당 블록을 삽입한다. applySw 함수와 searchquery 함수를 통해 수행하며, 이들을 통해 얻은 인덱스에 블록을 삽입하는 형태로 재구성을 한다.

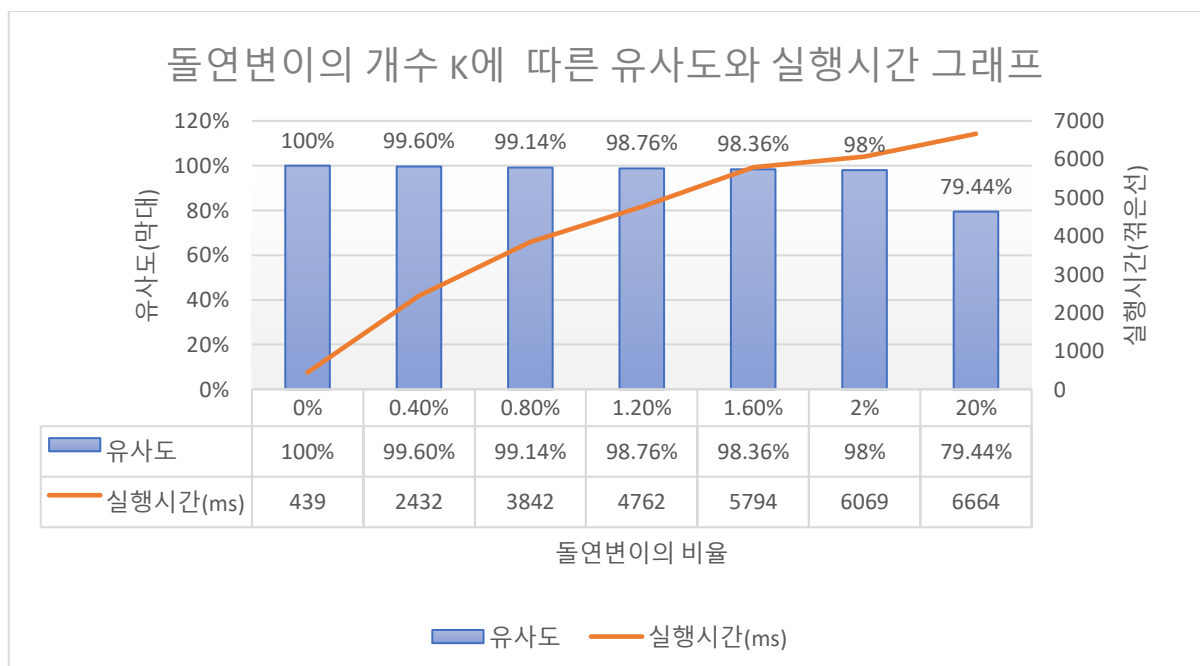
3.2.5. 주요 데이터 값에 따른 결과 비교

초기 조건은 돌연변이의 개수 $k=10$, short read 길이 $L=100$, sequence 길이 $n=5000$, short read 개수 $m=500$. 유사도의 조건은 레퍼런스(원본)시퀀스와 얼마나 유사한 가이다. 재구성된 시퀀스가 레퍼런스와 완전히 동일하다면 제대로 복원된 것이 아니므로, 이를 구별하기 위해 돌연변이 시퀀스가 아닌 레퍼런스와 비교했다. 따라서 돌연변이가 2%이면 최대 복원 가능 정도는 98%가 된다.

① 돌연변이의 개수 K

이 프로그램은 매칭을 위해 알고리즘을 2개 사용한다고 볼 수 있는데, 이 중 Smith Waterman 알고리즘의 경우 정확도는 높으나 매우 느리다. FM-Index Backward searching은 속도가 매우 빠르나, 돌연변이 등의 오차 발견시에 작동하지 않는다. 때문에 돌연변이의 개수가 실행시간에 큰 영향을 미치며 실행시간과 거의 정비례한다.

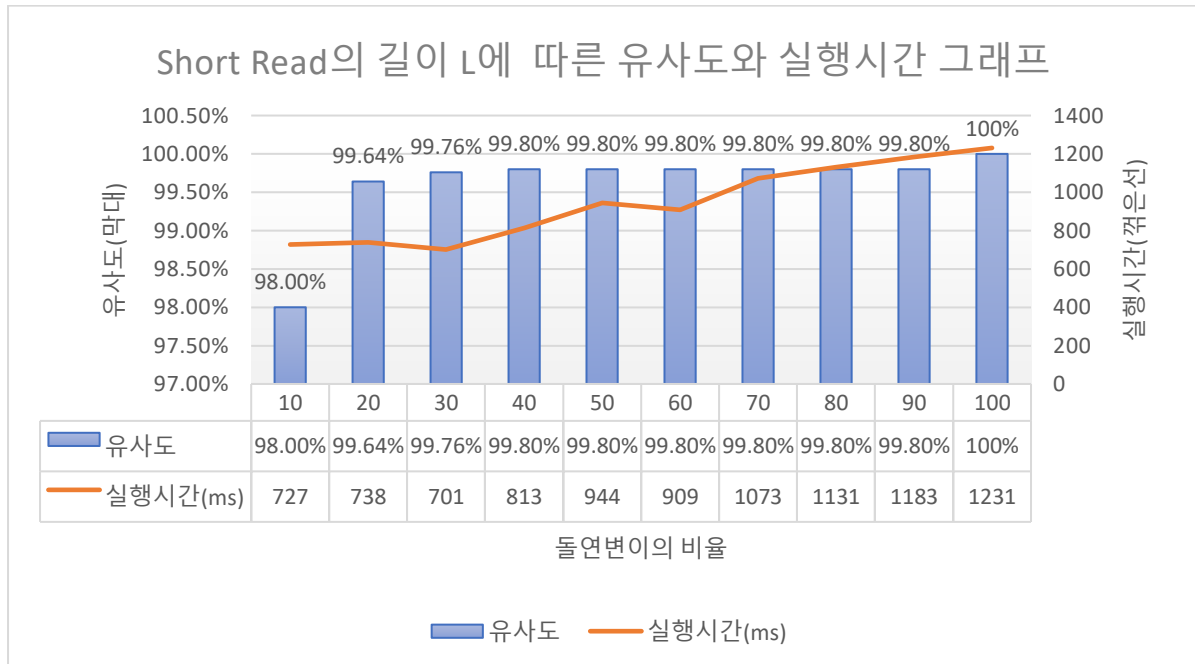
한편 유사도는 돌연변이의 개수와 상관없이 최고 수준을 유지하였는데, 돌연변이의 개수를 0개, 10개, 100개, 1000개로 증가시키면 따라 돌연변이의 비율은 0%, 0.2%, 2%, 20%로 증가하였으며, 이에 유사도는 돌연변이의 비율을 제외한 100%, 99.8%, 98%에 근사하게 낮아졌다.



[그래프 3.2-1] 돌연변이의 개수 K에 따른 유사도와 실행시간 그래프

② Short Read(블록)의 길이 L

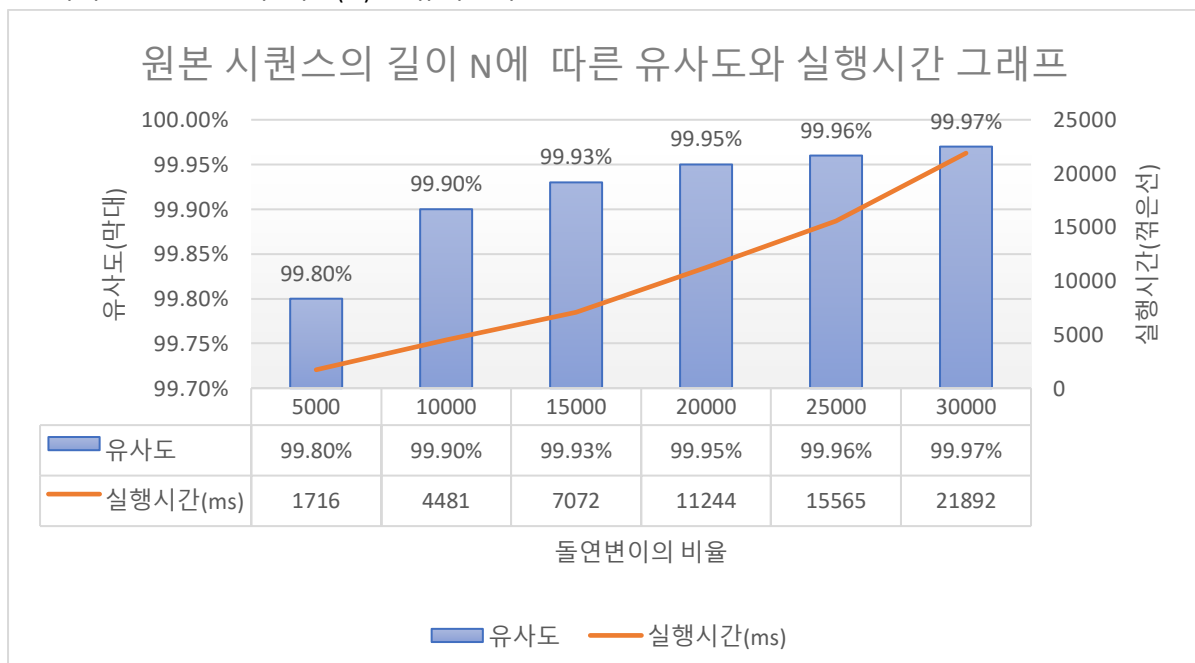
블록의 길이가 매우 짧은, 길이가 10 인 경우에도 98%의 높은 유사도가 나왔으며, 길이가 40 을 넘자 오차가 없었다. 실행시간은 쇼트리드의 길이에 비례했으나 돌연변이의 개수에 비해 완만한 증가폭을 가졌다.



[그래프 3.2-2] Short Read 의 길이 L 에 따른 유사도와 실행시간 그래프

③ 원본 시퀀스의 길이 N

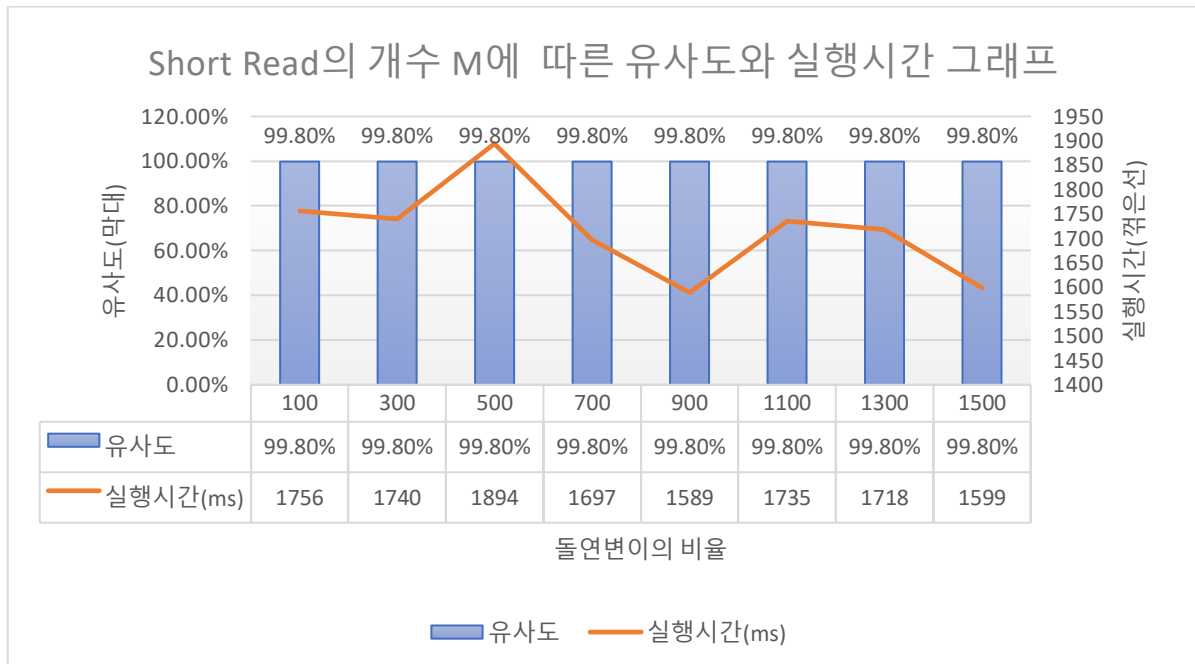
시퀀스의 길이 N이 증가함에 따라 실행시간도 가파르게 증가한다. 정비례하지 않는다. 유사도는 역시 100-돌연변이 비율(%)을 유지한다.



[그래프 3.2-3] 원본 시퀀스의 길이 N 에 따른 유사도와 실행시간 그래프

④ 블록 개수 M

알고리즘 수행시, 시퀀스를 전부 채우면 알고리즘이 종료되므로, 블록의 개수는 성능에 아무 영향이 없다.



[그래프 3.2-4] Short Read의 개수 M에 따른 유사도와 실행시간 그래프

3.2.6. 알고리즘의 장단점 및 개선방안

위의 수행 결과에 따라 BWT-SW 알고리즘의 장점과 단점은 아래와 같다. 우선 장점으로 FM-Index 를 이용한 빠르고 효율적인 검색과, Smith Waterman 알고리즘을 이용해서 돌연변이가 존재하는 경우에도 효율적인 검색이 가능하다는 점, 즉 정확하고 빠른 블록 검색이 가능하다는 것이다. Smith Waterman 알고리즘은 지역 정렬을 사용해 부분적인 일치도 감지 가능합니다. 이는 높은 돌연변이율과 적은 블록 개수에도 빠르고 정확하게 작동한다. 또한 비교적 작은 공간 복잡도를 가진다.

단점으로는 역시 BWT 구현과 SW 알고리즘의 사용에서 시간 복잡도가 크다는 점, 돌연변이의 개수가 늘어날수록 비효율적으로 작동하는 SW 알고리즘이 알고리즘의 전체 시간 복잡도를 크게 증가시킨다는 점 그리고 여러 알고리즘이 이용되어 코드가 복잡하고 이에 구현과 디버깅 등이 어려워진다는 점이다.

이를 개선하기 위해서는 여러 방안이 이용될 수 있는데, 우선 BWT 의 구현 과정에서 시간복잡도를 줄이는 방안이다. **선형 시간 접미사 배열 알고리즘 적용**할 수 있다. 현재 알고리즘에서는 접미사 배열을 활용하여 시간복잡도 $O(n \log n)$ 에 정렬을 수행하고 있는데, 정렬 방식을 카운팅 정렬이나 계수 정렬로 바꾸어 시간복잡도를 더 줄일 수도 있을 것이다. SW 알고리즘을 최적화할 수도 있을 것이다. Wavelet Tree 등의 효율적인 자료 구조를 사용하여 FM-Index 의 검색 성능을 향상시킬 수도 있다. GPU 를 이용하거나 스트리밍을 사용해 블록별로 병렬처리를 수행할 수도 있다. 아예 Smith waterman 없이, FM-Index 에서 일정 수준의 돌연변이를 허용하면서 검색할 수 있을 것이다.

3.3. Greedy Overlap Algorithm

3.3.1. 작동 원리

Greedy Overlap Algorithm 은 각 서열 조각(블록) 간의 최대 중첩(overlap)을 탐색하여 가장 긴 중첩을 가진 블록을 반복적으로 선택하는 방식으로 원래의 서열을 재구성한다.

3.3.2. 구성 요소

Greedy Overlap Algorithm 은 매개변수로 Short Reads(블록), 원본 시퀀스와 중첩 길이를 받는다. 블록은 ShortBlockGenerator 에서 생성된 섞인 블록들이 입력된다. 이 블록들은 중첩(overlap)을 활용하여 시퀀스를 재구성하는 데 사용된다. 블록들은 길이와 중첩 설정에 따라 생성되며, 섞인 상태로 알고리즘에 전달된다. 원본 시퀀스의 길이는 재구성된 시퀀스의 목표 길이를 결정하는 데 사용되고, 중첩 길이는 두 블록 간 최대 중첩 길이를 나타내며 이 값은 중첩을 계산하는 데 사용된다.

3.3.3. 알고리즘의 흐름

① 중첩 계산 (computeOverlap)

중첩 계산은 두 문자열 간의 가장 긴 일치 구간을 찾는 핵심 단계이다. 문자열 a의 끝부분과 문자열 b의 앞부분을 비교하고, i의 값을 줄여가며 최대 중첩 길이를 탐색한다. 중첩이 발견되면 i를 반환하고, 발견되지 않으면 0을 반환한다.

1. 문자열 a의 끝부분과 문자열 b의 앞부분을 비교한다.
2. i의 값을 줄여가며 최대 중첩 길이를 탐색한다.
3. 중첩이 발견되면 i를 반환하고, 발견되지 않으면 0을 반환한다.

② 시퀀스 복원 함수 (reconstructSequence)

중첩 계산 결과를 바탕으로 가장 긴 중첩을 가진 블록을 선택해 반복적으로 서열을 확장한다.

1. 초기화: 초기 단계에서 첫 번째 블록을 결과 시퀀스(result)에 추가한다.
2. 반복 과정: 현재 결과 문자열의 마지막 부분과 중첩이 최대인 블록을 찾는다. 선택된 블록의 중첩된 부분을 제외한 나머지 서열을 결과 문자열에 추가한다. 이 과정을 반복하여 블록들을 하나씩 추가한다. 이때, 사용된 블록을 used 집합에 기록하여 더 이상 사용되지 않게 한다.
3. 종료 조건: 더 이상 사용할 블록이 없거나, 결과 시퀀스(result)의 길이가 원본 시퀀스의 길이에 도달하면 알고리즘을 종료한다.

3.3.4. 알고리즘의 장점

① 간단한 구현과 이해 용이성: 알고리즘이 직관적이며 구현이 비교적 간단하다. 블록 간의 중첩만을 고려하여 순서를 결정한다. 또, 복잡한 데이터 구조나 사전 구축 과정 등과 같은 전처리가 비교적 적게 필요하다.

② 부분적인 중첩 활용 가능: 블록 간의 완전한 일치가 아닌 부분적인 중첩도 활용하여 시퀀스를 연결한다.

3.3.5. 결과 값 실험 및 분석

① 기준 값 결과 분석

<pre>// 기준 값으로 실험 int K = 50; // 돌연변이 개수 (20~100 중 50 선택) int L = 50; // 블록 길이 (32~100 중 50 선택) int N = 500000; // 파일에서 읽어올 알파벳 수 (100,000 ~ 3,000,000,000 중 중간값인 500,000 선택) int M = 5000; // 블록의 개수 (20million~200million 중 중간값을 선택) int J = 40; // 중복되는 길이</pre>	Greedy Overlap Algorithm 실행 : 알고리즘 수행 시간 : 36784ms 매칭 완료, 분석 수행 돌연변이의 비율 : 1% 유사도 : 68.44%
---	---

[그림 3.3-1] 기준 값 설정 및 실행 결과

결과 분석

1. 수행 시간이 긴 이유

- 블록 개수와 데이터 크기의 영향: N=500,000 과 M=5,000 의 설정으로 인해 알고리즘이 비교해야 할 데이터 블록의 조합이 많아졌다. 각 블록마다 매칭을 수행해야 하므로 수행 시간이 증가했다. 특히, 탐색 공간에서 블록 간 매칭을 반복적으로 수행하기 때문에 연산량이 선형적으로 증가했을 가능성이 있다.

- 중복 데이터 매칭 부담: J=40 으로 중복 데이터가 설정되어 있어 블록 간 매칭 과정에서 비교 작업이 추가되었다. 하지만 돌연변이 비율(1%)이 낮아 매칭 실패로 인한 추가 작업은 많지 않았다.

2. 유사도가 높은 이유

블록 간 데이터 유사도가 68.44%로 높은 것은 설정된 중복 데이터(J=40)가 주요 원인으로 보인다. 이는 데이터 블록에서 공통된 패턴이나 연속된 데이터가 많다는 것을 암시하며, 알고리즘이 효율적으로 중복 데이터를 탐지했음을 나타낸다.

3. 돌연변이 비율이 낮은 이유

돌연변이 비율이 1%로 매우 낮기 때문에, 알고리즘 수행 시간에 돌연변이가 기여하는 영향은 미미했다. 이는 데이터 처리에 있어 주로 중복 데이터 매칭 및 탐색이 성능에 영향을 미쳤음을 시사한다.

② 복잡도 증가할 경우

<pre>// 복잡도 증가할 경우 한계 실험 int K = 80; // 돌연변이 개수 (20~100 중 80 선택) int L = 100; // 블록 길이 (32~100 중 최대값인 100 선택) int N = 3000000; // 파일에서 읽어올 알파벳 수 (100,000 ~ 3,000,000,000 중 최대값인 3,000,000 선택) int M = 20000; // 블록의 개수 (20million~200million 중 큰 값인 20,000 선택) int J = 60; // 중복되는 길이 (기준 값에서 증가시킴)</pre>	Greedy Overlap Algorithm 실행 : 알고리즘 수행 시간 : 2884ms 매칭 완료, 분석 수행 돌연변이의 비율 : 1.6% 유사도 : 72.96%
--	--

[그림 3.3-2] 복잡도 증가할 경우 실행 결과

결과 분석

1. 알고리즘 수행 시간: 2,884ms

수행 시간이 2.88초로, 기준 값 실험(36.7초) 대비 매우 빠르게 감소하였다. 이는 복잡도 증가에도 불구하고, 알고리즘이 중복 데이터의 패턴을 효율적으로 매칭하여 데이터 처리를 최적화하였기 때문이다. 또, 블록 길이(L)와 중복 길이(J)가 증가하면서 데이터가 연속적으로 나타나 매칭 과정에서 중복 블록의 탐색 효율성이 높아졌을 가능성도 있다.

2. 유사도: 72.96%

유사도가 72.96%로 기준 실험(68.44%)보다 증가했다. 이는 블록 길이(L=100)와 중복 길이(J=60)가 늘어남에 따라, 데이터 간 매칭 성공률이 높아졌음을 나타낸다.

3. 돌연변이 비율: 1.6%

돌연변이 비율이 기준 실험(1%)보다 소폭 증가했다. 이는 돌연변이로 인한 매칭 실패 비율이 소폭 증가했음을 의미하지만, 여전히 전체 매칭 과정에 큰 영향을 주지 않았음을 시사한다.

복잡도가 증가한 변수에 따른 분석

1. 파일 크기(N=3,000,000)

데이터 크기를 최대값으로 설정했음에도 수행 시간이 크게 증가하지 않은 것은, 알고리즘이 데이터를 처리하는 방식이 파일 크기 증가에 선형적으로 영향을 받지 않았음을 보여준다.

2. 블록 개수(M=20,000)

블록 개수가 증가하면 탐색 공간이 커지므로, 일반적으로 수행 시간이 늘어날 가능성이 크다. 그러나 본 실험에서는 중복 데이터의 패턴(J=60)을 활용하여 탐색 과정을 최적화하였다.

3. 블록 길이(L=100)

블록 길이가 최대값으로 설정됨에 따라, 각 블록이 포함하는 데이터의 양이 많아졌다. 이는 매칭에 필요한 연산을 줄이고, 탐색 과정에서의 효율성을 높였다.

4. 중복 길이(J=60)

중복 길이가 늘어나면서, 블록 간 매칭 성공 확률이 높아졌다. 이는 유사도 증가(72.96%)로 나타났다으며, 매칭 과정에서 불필요한 연산이 줄어들었기 때문이다.

③ 복잡도 낮아질 경우

// 복잡도 낮은 경우 한계 실험	Greedy Overlap Algorithm 실행:
int K = 20; // 돌연변이 개수 (20~100 중 최소값인 20 선택)	알고리즘 수행 시간: 15596ms
int L = 32; // 블록 길이 (32~100 중 최소값인 32 선택)	
int N = 100000; // 파일에서 읽어들인 알파벳 수 (100,000 ~ 3,000,000,000 중 최소값인 100,000 선택)	매칭 완료, 분석 수행
int M = 1000; // 블록의 개수 (20million~200million 중 작은 값인 1,000 선택)	돌연변이의 비율: 0.4%
int J = 20; // 중복되는 길이 (기준 값보다 적게 설정)	유사도: 76.36%

[그림 3.3-3] 복잡도 낮아질 경우 실행 결과

결과 분석

1. 알고리즘 수행 시간: 15,596ms

복잡도가 낮은 상황에서도 수행 시간이 기준 실험(36,784ms)보다 절반 이하로 감소하였다. 복잡도가 낮아졌음에도 수행 시간이 복잡도가 높은 실험(2,884ms)보다 길어진 이유는 다음과 같다.

- 데이터 패턴 제한: 짧은 블록 길이(L=32)와 중복 길이(J=20)는 매칭 알고리즘이 데이터 패턴을 탐지하기 어렵게 만든다.

- 탐색 공간 감소: 블록 개수(M=1,000)와 데이터 크기(N=100,000)가 작아 탐색할 데이터 양이 줄어들었지만, 이로 인해 유효한 매칭 블록을 찾는 데 추가적인 연산이 발생했을 가능성이 있다.

2. 돌연변이 비율: 0.4%

돌연변이 비율이 기준 실험(1%)이나 복잡도가 높은 실험(1.6%)보다 낮다. 이는 돌연변이 개수(K=20)가 작아져, 매칭 과정에서 돌연변이에 의해 실패하는 경우가 줄어들었음을 의미한다. 또, 돌연변이의 비율 감소는 유사도 증가에 기여했을 가능성이 크다.

3. 유사도: 76.36%

유사도가 기준 실험(68.44%)과 복잡도가 높은 실험(72.96%)보다 증가했습니다. 이는 블록 크기가 감소함(L=32)에 따라, 짧은 블록 크기로 인해 작은 데이터 패턴도 매칭이 성공했을 가능성이 높다. 또, 데이터 크기가 작아(N=100,000) 유효 매칭률이 상대적으로 높아졌다.

복잡도가 증가한 변수에 따른 분석

1. 파일 크기(N=100,000)

파일 크기가 작기 때문에 탐색 공간이 제한적이었으나, 이는 오히려 알고리즘이 과도한 데이터를 처리하지 않아 연산 효율을 유지하는 데 기여하였다.

2. 블록 길이(L=32)

짧은 블록 길이는 유사도 측면에서는 긍정적인 영향을 주지만, 알고리즘이 효율적으로 동작하기 위해 필요한 특징 패턴을 감소시켜, 연산 시간이 늘어났을 가능성이 있다.

3. 중복 길이(J=20)

짧은 중복 길이는 패턴 매칭에서 애매한 결과를 초래할 가능성이 있다. 중복 길이가 짧아 탐색 공간이 줄어들었으나, 연산 과정에서 비교 연산이 늘어났을 수 있다.

4. 돌연변이 개수(K=20)

돌연변이 개수가 적으므로, 매칭 실패 요인 자체가 줄어들었다. 그러나 낮은 돌연변이 개수는 알고리즘의 돌연변이 탐지 성능을 충분히 시험하지 못했을 가능성이 있다.

실험	수행 시간	돌연변이 비율	유사도	주요 변수 설정
기준 실험	36,784ms	1.0%	68.44%	K=40, L=64, N=1,000,000, M=10,000, J=30
복잡도 증가	2,884ms	1.6%	72.96%	K=80, L=100, N=3,000,000, M=20,000, J=60
복잡도 감소	15,596ms	0.4%	76.36%	K=20, L=32, N=100,000, M=1,000, J=20

[표 3.3-1] 실험 및 분석 결과 정리 표

3.3.6. 주요 데이터 값에 따른 결과 비교

① 원본 시퀀스(파일)의 길이 N

- 기준 실험: N=1,000,000 → 복잡도 증가: N=3,000,000

데이터 크기가 3 배로 증가했음에도 불구하고, 중복 블록 길이(J)가 커지면서 알고리즘의 매칭 효율이 높아졌다. 이로 인해 불필요한 탐색이 줄어들어 수행 시간이 감소(2,884ms)했으며, 유사도(72.96%)도 증가하였다. 그 결과, 파일 크기 증가와 관계없이 중복 블록 길이 증가로 인해 수행 시간이 감소하고 유사도는 증가했다.

- 기준 실험: N=1,000,000 → 복잡도 감소: N=100,000

데이터 크기를 줄였지만, 중복 블록 길이(J)가 작아지면서 매칭 효율이 떨어졌다. 중복 영역이 적어 탐색 공간이 넓어지고 연산량이 증가한 결과, 수행 시간이 15,596ms 로 증가하였다. 즉, 데이터 크기 감소는 영향을 미치지 않았으며, 중복 블록 길이 감소로 인해 수행 시간이 증가했다.

② Short Read(블록)의 개수 M

- 기준 실험: M=10,000 → 복잡도 증가: M=20,000

블록 개수가 많아지면 더 많은 블록을 매칭해야 하지만, 데이터 패턴이 명확한 조건(N=3,000,000, L=100) 덕분에 수행 시간은 단축되었다.

- 기준 실험: M=10,000 → 복잡도 감소: M=1,000

블록 개수가 적어지면 데이터 패턴의 다양성이 제한된다. 이로 인해 수행 시간이 길어지는 현상이 발생하였다.

③ Short Read(블록)의 길이 L

- 기준 실험: $L=64 \rightarrow$ 복잡도 증가: $L=100$

블록 길이가 증가하면서 데이터의 매칭 패턴이 더 명확해지고, 유사도(72.96%)가 증가하였다. 이는 중복 길이($J=60$)와도 조화를 이루며 더 높은 매칭 성공률로 이어졌다.

- 기준 실험: $L=64 \rightarrow$ 복잡도 감소: $L=32$

블록 길이가 짧아지면 매칭 과정에서 비교 연산이 더 많아지고, 데이터 패턴이 희미해져 수행 시간이 늘어났다. 하지만 상대적으로 작은 데이터에서 짧은 블록 길이는 유사도(76.36%) 상승에 기여하였다.

④ 중복(overlap)의 길이 J

- 기준 실험: $J=30 \rightarrow$ 복잡도 증가: $J=60$

중복 길이가 길어지면서 패턴 매칭이 더 효과적으로 이루어져 유사도가 증가하였다. 긴 중복 길이는 수행 시간 단축에도 기여하였다.

- 기준 실험: $J=30 \rightarrow$ 복잡도 감소: $J=20$

중복 길이가 짧아지면서 비교 연산이 늘어나 수행 시간이 증가했고, 매칭 패턴이 불확실해져 알고리즘의 효율성이 감소하였다.

⑤ 돌연변이의 개수 K

- 기준 실험: $K=40 \rightarrow$ 복잡도 증가: $K=80$

돌연변이 개수가 많아지면 매칭 과정에서 추가적인 돌연변이 탐색 연산이 요구된다. 이로 인해 돌연변이 비율(1.6%)이 증가했으나, 데이터 패턴이 커져 돌연변이 탐색의 영향은 유의미한 차이를 만들지 않았다. 그 결과, 돌연변이 개수 증가에도 불구하고, 수행 시간이 감소하고 유사도는 증가했다. 이는 돌연변이 개수가 많아진 것보다 파일 크기(N)가 커졌을 때 효율적으로 처리된 결과로 해석할 수 있다.

- 기준 실험: $K=40 \rightarrow$ 복잡도 감소: $K=20$

돌연변이 개수를 줄이면 돌연변이 탐지가 줄어들고 매칭이 쉬워져 돌연변이 비율은 낮아지지만, 파일 크기(N)가 작아졌기 때문에 돌연변이의 상대적인 비율이 자연스럽게 낮아졌다. 이로 인해 유사도가 높아졌고, 수행 시간은 증가했다. 그 결과, 돌연변이 개수 감소 외에도 파일 크기 감소가 돌연변이 비율을 낮추고 유사도를 높였다는 점에서, 파일 크기(N)의 영향이 더 크게 작용한 것으로 분석된다.

결론 및 비교 분석

수행 시간 변화의 주된 원인은 파일 크기(N)의 변화가 아니라 중복 블록 길이(J)의 변화이다. 또, J 가 커질수록 매칭 효율이 증가해 탐색 공간이 줄어들며, 불필요한 연산이 감소한다. 따라서 실험 결과는 중복 블록 길이(J)가 알고리즘 효율에 미치는 영향력을 보여준다.

4. 결과 비교 및 분석

4.1. 알고리즘별 시공간 복잡도

세 알고리즘인 DeBruijn Graph, BWT_SW, Greedy Overlap 알고리즘의 시간 및 공간 복잡도를 계산하고 비교해보겠다. 이를 통해 각 알고리즘의 장단점과 특성을 이해할 수 있다.

4.1.1. DeBruijn Graph Algorithm

DeBruijn Graph 알고리즘에서 복잡도에 영향을 미치는 변수는 블록의 개수 M , 블록의 길이 L 과 k -mer 의 k 의 값이다. 크게 두 과정, 그래프를 생성하는 bulidGraph 와 오일러 경로를 탐색하는 findEulerianPath 에서 비용이 발생한다.

bulidGraph 에서 시간 복잡도는 $O(M \cdot L \cdot K)$, 공간 복잡도는 $O(M \cdot L)$ 이고, findEulerPath 에서 시간복잡도는 $O(M \cdot L)$, 공간 복잡도는 $O(M \cdot L)$ 이다.

단계	수행 작업	시간복잡도	공간복잡도
bulidGraph	그래프 생성	$O(M \cdot L \cdot k)$	$O(M \cdot L)$
k-mer 생성	각 Short Read에서 k-mer 생성	$O(M \cdot L \cdot k)$	$O(M \cdot L)$
addEdge	노드(prefix와 suffix)와 간선 추가	$O(M \cdot L \cdot k)$	$O(M \cdot L)$
findEulerianPath	오일러 경로를 탐색	$O(M \cdot L)$	$O(M \cdot L)$
시작/종료 노드 찾기	진입/진출 차수 계산, V =노드 수 $O(M \cdot L)$, E =간선 수 $O(M \cdot L)$	$O(V+E) = O(M \cdot L)$	$O(M \cdot L)$
Euler Path 탐색	깊이 우선 탐색(DFS) 기반	$O(E) = O(M \cdot L)$	$O(M \cdot L)$
reconstructSequence	서열 복원, 복원된 서열을 저장	$O(M \cdot L \cdot k)$	$O(M \cdot L + N)$

[표 4-1] DeBruijn Graph 알고리즘의 단계별 시공간 복잡도

따라서, 총 시간 복잡도는 $O(M \cdot L \cdot k)$, 공간 복잡도는 $O(M \cdot L + N)$ 이다.

4.1.2. BWT-SW

BWT-SW 알고리즘에서 주요 변수는 원본 시퀀스의 길이 N , 블록의 개수 M , 블록의 평균 길이 L , 돌연변이의 개수 K 이다.

주된 알고리즘은 BWT 적용, FM-Index 구축, Backward Searching, SW 알고리즘 이렇게 4가지이다. BWT의 시간 복잡도는 $O(N \log^2 N)$, 공간 복잡도는 $O(N)$ 이며, FM-Index 구축 과정에서는 시간 복잡도가 $O(N)$, 공간 복잡도가 $O(5N) = O(N)$ 이다. 블록의 검색 과정에서의 시간 복잡도는 Backward Searching에서 $O(M \times L)$, Smith-Waterman에서 $O(K \times N \times L)$ 이다. Backward searching 과정에서는 FM-Index에서 구축한 공간 복잡도를 유지하고, Smith-Waterman은 수행 시 $O(N \times L)$ 이다.

단계	수행 작업	시간복잡도	공간복잡도
Burrows-Wheeler Transform (BWT)	원본 시퀀스 → BWT 형식으로 변환	$O(N \log^2 N)$	$O(N)$
BWT: 접미사 배열 생성 및 정렬	-	$O(N \log^2 N)$	$O(N)$
BWT: 문자열 생성	정렬된 접미사 배열을 shift하며 생성	$O(N)$	$O(N)$

FM-Index 구축	5는 문자의개수, ATGC\$	$O(5N) = O(N)$	$O(5N) = O(N)$
C 배열 생성	각 문자의 누적빈도 계산	$O(N)$	$O(5) = O(1)$
Occ 배열 생성	위치별 문자별 누적 개수 저장	$O(5N) = O(N)$	$O(5N) = O(N)$
블록 검색 과정	Backward Searching+ SW	$O(ML + KML)$	FM-Index+ $O(N \times L)$
FM-Index Backward Search	-	$O(L)$	$O(L)$
Smith-Waterman 알고리즘	돌연변이 존재 블록에 대해 지역 정렬 수행	$O(K \times N \times L)$	$O(N \times L)$

[표 4-2] BWT-SW 알고리즘의 단계별 시공간 복잡도

따라서, 총 시간 복잡도는 $O(N \log^2 N + ML + KML)$, 공간 복잡도는 $O(N \times L)$ 이다.

4.1.3. Greedy Overlap Algorithm

Greedy-Overlap 알고리즘은 각 블록 쌍의 중첩 길이를 계산(computeOverlap)하고, 이 중에서 중첩 길이가 가장 긴 블록을 선택해 서열을 복원(reconstructSequence)한다.

두 문자열의 중첩을 계산하는 비용은 $O(L)$, 총 블록 쌍은 $M(M-1)$ 개이므로 전체 중첩 길이 계산 비용은 $O(M^2 \cdot L)$ 이다. 서열 복원 단계에서 최적의 블록을 선택하는 데 $O(M)$ 을 소요하고, 총 M 개의 블록을 처리해야 하므로 전체 서열 복원 비용은 $O(M^2)$ 이다. 그러므로 총 시간 복잡도는 $O(M^2 \cdot L) + O(M^2) = O(M^2 \cdot L)$ 이다.

중첩 길이 계산 단계에서 $M \times M$ 의 중첩 정보를 저장하므로 $O(M^2)$, 서열 복원 단계에서 결과를 저장하는데 $O(N)$ 을 소요하므로 총 공간 복잡도는 $O(M^2 + N)$ 이다.

단계	수행 작업	시간복잡도	공간복잡도
중첩 계산 (computeOverlap)	각 블록 쌍의 중첩 길이 계산	$O(M^2 \cdot L)$	$O(M^2)$
서열 복원 (reconstructSequence)	최적의 블록을 선택하여 서열 복원	$O(M^2)$	$O(N)$
전체	중첩 계산 + 서열 복원	$O(M^2 \cdot L)$	$O(M^2 + N)$

[표 4-3] Greedy Overlap 알고리즘의 단계별 시공간 복잡도

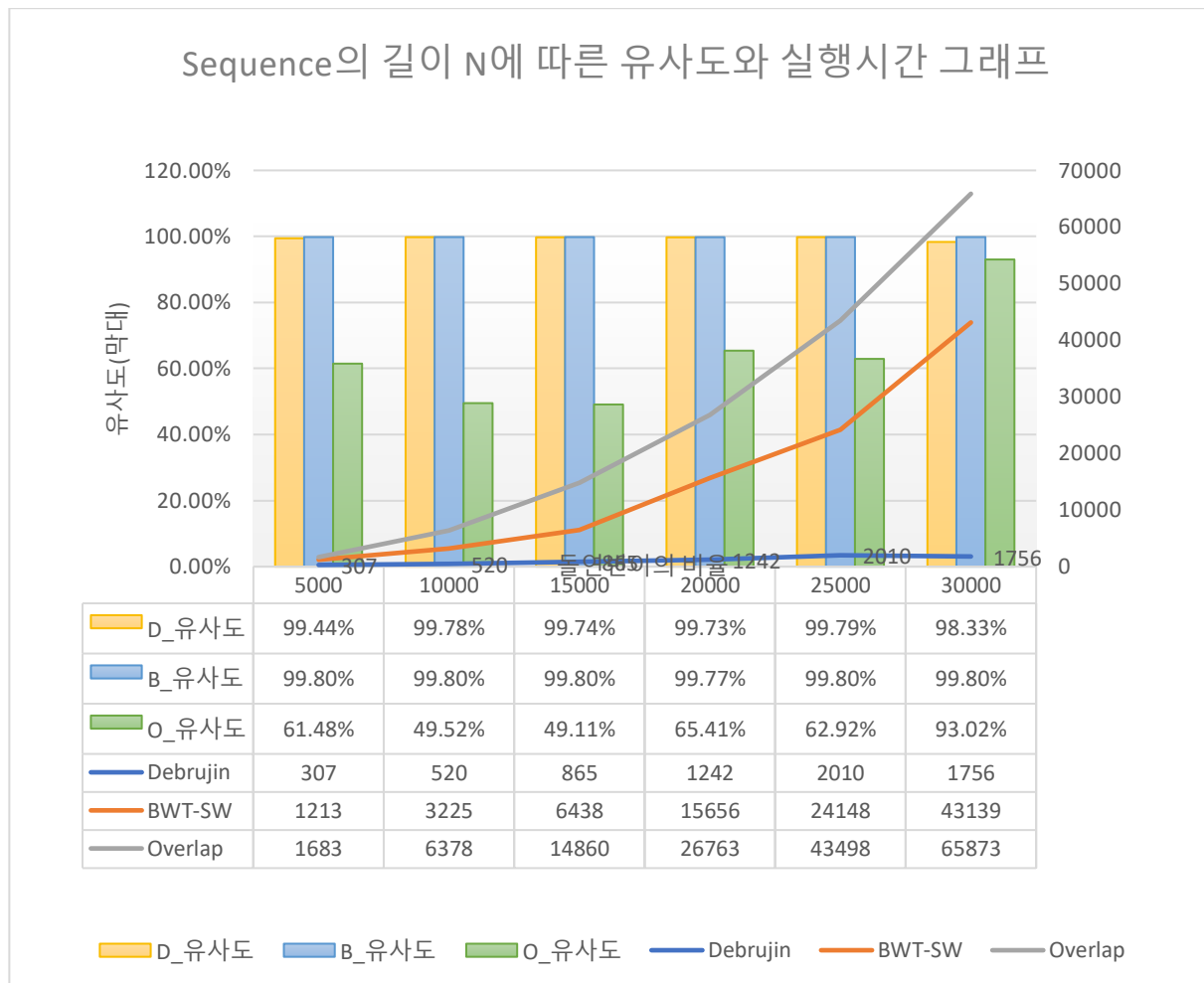
4.1.4. 비교 표

다음은 앞에서 계산한 세 알고리즘 DeBruijn Graph, BWT-SW, Greedy Overlap 의 시간복잡도와 공간복잡도를 정리한 표이다.

알고리즘	시간복잡도	공간복잡도
DeBruijn Graph	$O(M \cdot L \cdot k)$	$O(M \cdot L + N)$
BWT-SW	$O(N \log^2 N + ML + KML)$	$O(N \times L)$
Greedy Overlap	$O(M^2 \cdot L)$	$O(M^2 + N)$

[표 4-4] 세 알고리즘의 시공간 복잡도를 비교한 표

4.2. 실제 프로그램 작동 시 비교



[그래프 4-1] Sequence의 길이 N에 따른 유사도와 실행시간 그래프

다른 조건을 유지하고, 블록의 개수 M 을 N 에 대해 20%비율, 블록의 길이를 100, 돌연변이의 비율을 0.2%로 유지하여 N 을 증가시킨 결과이다.

1. Debrujin Graph 알고리즘: 안정적인 시간복잡도 증가율을 보인다. 매우 빠르게 매칭이 수행되며, m 의 비율이 높은 위의 경우 안정적인 수행 결과를 보인다.
2. BWT-SW 알고리즘: 가파른 시간복잡도 증가율을 보인다. 이는 돌연변이의 비율을 고정하여 N 이 커질수록 돌연변이의 개수도 늘어났기 때문이다. 매우 안정적인 유사도를 보장한다.
3. Greedy Overlap 알고리즘: 셋 중 가장 높은 시간 복잡도 증가율과, 낮은 유사도를 보인다. 블록의 개수를 고정 비율로 정리하여, Overlap 수행이 그에 비례하여 증가하였기 때문이다.

추가적으로, 원본 시퀀스 길이 N 이 지나치게 클 경우 유사도가 낮게 산출되거나 처리 시간이 과도하게 소요되는 문제가 확인되었다. 이를 해결하기 위해 데이터 구조를 최적화하고 알고리즘 복잡도를 개선하여 N 이 수백만에 달하더라도 효율적으로 처리할 수 있도록 개선할 예정이다.

5. 결론

1. DeBruijn Graph: 대규모 데이터에서 De Bruijn Graph는 매우 효율적이며, 적절한 데이터 설정이 필수이다. 데이터가 충분히 연결되지 않으면 문제가 발생하므로 이를 해결하기 위한 재탐색 알고리즘이 필요하다. 주요 요소로는 블록의 개수 M , 블록의 길이 L , k-mer 크기 k 가 있다.

2. BWT_SW: 시퀀스 길이 N , 블록 개수 M , 블록 길이 L , 돌연변이 개수 K 가 주요 요소이며 여러 요소의 변화에 안정적으로 높은 정확도를 보이지만, 돌연변이가 많은 서열에서 BWT-SW가 속도가 느리다. Smith-Waterman 알고리즘을 최적화하거나 병렬처리를 통해 성능을 개선할 수 있다.

3. Greedy Overlap: 블록 개수 M , 블록의 길이 L , 데이터 중복 개수 J 가 주요 요소이며, 정확도가 떨어진다. 간단한 서열 복원이 필요하다면 Greedy Overlap이 적합하지만, 정확도를 많이 요구하는 문제나 대규모 데이터에서는 최적화가 필요하다.

각 알고리즘은 상황에 따라 장점과 단점이 명확하므로, 필요에 따른 알고리즘 선택이 중요하며, 현재의 시간 복잡도로는 데이터 처리에 한계가 있으므로, 대규모 데이터에서 알고리즘을 최적화할 필요가 있다.

다음은 최종 성능 평가용 데이터를 입력했을 때의 결과이다.

```
int K = 60; // 돌연변이 개수
int L = 100; // 블록 길이
int N = 30000; // 파일에서 읽어들 알파벳 수
int M = 6000; // 블록의 개수
int J = 40; // 중복되는 길이

DeBruijn Graph Algorithm 실행:
알고리즘 수행 시간: 2543ms
매칭 완료, 분석 수행
돌연변이의 비율: 0.2%
유사도: 99.7867%

BWT+SW Algorithm 실행:
알고리즘 수행 시간: 51564ms
매칭 완료, 분석 수행
돌연변이의 비율: 0.2%
유사도: 99.7933%

Greedy Overlap Algorithm 실행:
알고리즘 수행 시간: 64781ms
매칭 완료, 분석 수행
돌연변이의 비율: 0.2%
유사도: 99.0167%
```

[그림 5-1] 최종 성능 평가용 데이터 입력 및 결과

6. 참고문헌

- (1) Compeau, P. E., Pevzner, P. A., & Tesler, G. (2011). Why biologists need algorithms. *Nature Biotechnology*.
- (2) Burrows, M., & Wheeler, D. J. (1994). A block-sorting lossless data compression algorithm.
- (3) National Center for Biotechnology Information (NCBI). (n.d.). Retrieved from <https://www.ncbi.nlm.nih.gov/>
- (4) Pevzner, P. A., Tang, H., & Waterman, M. S. (2001). An Eulerian path approach to DNA fragment assembly. *Proceedings of the National Academy of Sciences*, 98(17), 9748–9753. <https://doi.org/10.1073/pnas.171285098>
- (5) Holley, G., Wittler, R., & Stoye, J. (2020). Simplitigs as an efficient and scalable representation of de Bruijn graphs. *bioRxiv*. <https://doi.org/10.1101/2020.01.12.903443>
- (6) Smith, T. F., & Waterman, M. S. (1981). Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1), 195–197. [https://doi.org/10.1016/0022-2836\(81\)90087-5](https://doi.org/10.1016/0022-2836(81)90087-5)
- (7) Bowe, A. (n.d.). FM-Index. Retrieved from <https://www.alexbowe.com/fm-index/>